

Diseño y Programación de Plataformas

Móviles

I Ciclo 2026

Docente: Mag. Rachel Bolívar Morales

Examen I

Valor: 10%

Puntaje Total: 110

Fecha: 25 de Abril

Hora de Inicio: 8:15 a.m

Duración Máxima: 2 h y 30 minutos

Modalidad: Individual

1. CONTEXTO DE LA EVALUACIÓN

Usted ha sido contratado como desarrollador móvil por una empresa de tecnología que desea evaluar su capacidad para diseñar soluciones móviles de forma profesional.

Como parte del proceso de selección, se le solicita desarrollar una propuesta técnica integral para una nueva aplicación móvil, junto con una prueba de concepto (PoC) de la interfaz utilizando Jetpack Compose.

Esta evaluación no se enfoca únicamente en la implementación, sino en su capacidad para:

- Analizar un problema
- Diseñar una solución coherente
- Tomar decisiones técnicas fundamentadas
- Estructurar correctamente un sistema
- Comunicar sus ideas de forma clara y profesional

La entrega será evaluada como si fuera presentada a un equipo técnico real.

2. ENUNCIADO DEL PROBLEMA

La empresa desea desarrollar una aplicación móvil para la gestión de espacios de coworking, orientada a usuarios que necesitan encontrar y explorar espacios disponibles para trabajar.

La aplicación debe permitir:

- Visualizar un listado de espacios disponibles
- Mostrar información relevante de cada espacio, como:
 - Nombre
 - Imagen del espacio
 - Descripción
 - Ubicación
 - Capacidad
 - Precio por hora
 - Disponibilidad
- Permitir al usuario navegar hacia una vista de detalle de cada espacio
- Mostrar información ampliada del espacio seleccionado

- Poder realizar reservaciones de espacios

Alcance de la prueba de concepto

Para esta evaluación, el estudiante deberá diseñar la base del sistema y construir una interfaz funcional utilizando datos simulados.

No se requiere:

- Conexión con backend
- Persistencia de datos
- Implementación de lógica de negocio real

Enfoque esperado

El estudiante debe demostrar:

- Capacidad de traducir un problema a una solución técnica
- Diseño estructurado de la interfaz
- Uso adecuado de componentes reutilizables
- Organización del código siguiendo buenas prácticas

3. ESTRUCTURA DE LA ENTREGA

La entrega deberá realizarse en el repositorio asignado en GitHub Classroom, respetando estrictamente la siguiente estructura:

/app (proyecto Android con Jetpack Compose)

/docs

 /architecture

 /backend

 /api

 /ui

 /decisions

README.md

Lineamiento obligatorio de organización

- Cada sección debe incluir al menos un documento
- El nombre del archivo debe ser representativo del contenido
- Se permite el uso de formatos como .docx, .pdf, .png (diagramas), entre otros
- Todos los documentos deben estar ubicados en la carpeta correspondiente
- Cualquier documento que no se encuentre en la carpeta correspondiente no será evaluado.

4. CONDICIONES DE ENTREGA EN GITHUB CLASSROOM

La entrega del examen se realizará exclusivamente mediante el repositorio asignado en GitHub Classroom.

Se establecen las siguientes condiciones obligatorias:

- Todo el trabajo debe estar integrado en la rama **main**
- Todas las ramas utilizadas durante el desarrollo deben estar **correctamente mergeadas a main antes**

de la hora de entrega

- No se evaluarán cambios que no se encuentren en la rama main

Restricción de tiempo

- La hora de entrega será estrictamente respetada
- Cualquier commit realizado después de la hora límite de entrega no será considerado en la evaluación
- No se aceptarán entregas tardías bajo ninguna circunstancia

5. ENTREGABLES REQUERIDOS

5.1 Arquitectura del Sistema

Ubicación: /docs/architecture

El estudiante deberá presentar:

- Tipo de aplicación (nativa, híbrida, etc.)
- Patrón de arquitectura seleccionado
- Justificación técnica basada en el problema planteado
- Diagrama de arquitectura del sistema
- Explicación del flujo general del sistema

5.2 Diseño Backend (Conceptual)

Ubicación: /docs/backend

El estudiante deberá:

- Definir la arquitectura backend basada en capas
- Explicar el rol de cada capa
- Describir el flujo de una petición desde el cliente hasta la base de datos
- Justificar las decisiones tomadas

5.3 API Contracts

Ubicación: /docs/api

El estudiante deberá:

- Definir al menos dos endpoints PUT o POST relevantes para el sistema
- Especificar:
 - Método HTTP
 - Estructura de request
 - Estructura de responses
- Mantener coherencia con el diseño del sistema
- Presentar la información de forma estructurada (YAML, documento o tabla)

5.4 Implementación de UI (Jetpack Compose)

Ubicación: /app

El estudiante deberá implementar una prueba de concepto funcional de la interfaz utilizando Jetpack Compose.

Pantallas requeridas

- Pantalla de listado de espacios utilizando LazyColumn
- Pantalla de detalle de un espacio seleccionado
- Un componente de navegación tipo BottomBar visible en ambas pantallas

Datos

- Se deben utilizar datos simulados (mock data) definidos directamente en el código
- Los atributos de los datos deben ser definidos por el estudiante con base en el análisis del problema

Componentes obligatorios de Jetpack Compose

El estudiante deberá utilizar, como mínimo, los siguientes componentes nativos de Jetpack Compose:

- LazyColumn (para el listado)
- Card o Surface (para representar cada elemento del listado)
- Text (para mostrar información)
- Image (puede ser imagen simulada)
- Button o IconButton
- Scaffold (para estructurar la pantalla)
- BottomNavigation o NavigationBar (para el BottomBar)
- Row, Column y/o Box para organización de layouts
- Modifier para manejo de estilos y comportamiento

Requisitos obligatorios de implementación

- Uso exclusivo de Jetpack Compose
- Uso de datos simulados (mock data)
- Separación clara entre:
 - Screens (pantallas completas)
 - Components (elementos reutilizables)
- Implementación de componentes reutilizables
- Navegación básica entre pantallas (puede ser simulada)
- Código escrito en inglés
- Código limpio, legible y modular
- Organización coherente de archivos y carpetas

Restricciones

- No se permite lógica de negocio dentro de los composables
- No se permite implementar toda la interfaz en un solo composable
- No se permite código desorganizado o sin estructura clara

5.5 Diseño de UI

Ubicación: /docs/ui

El estudiante deberá documentar:

- Estructura de la interfaz implementada
- Lista de composables creados
- Identificación de componentes reutilizables
- Justificación de la organización de la interfaz

5.6 Decisiones Técnicas

Ubicación: /docs/decisions

El estudiante deberá desarrollar un análisis que incluya:

Propuesta de integraciones adicionales

- ¿Qué otras funcionalidades o integraciones recomendaría agregar a la aplicación (por ejemplo: notificaciones, autenticación, pagos, geolocalización, entre otros)?
- Justifique por qué estas integraciones aportarían valor al sistema

Mejoras en la interfaz de usuario

- ¿Qué mejoras realizaría a la UI implementada para mejorar la experiencia del usuario?
- Justifique sus decisiones considerando aspectos como usabilidad, claridad y organización

6. LINEAMIENTOS GENERALES DE EVALUACIÓN

Esta evaluación prioriza:

- Coherencia técnica del diseño
- Capacidad de análisis
- Justificación de decisiones
- Integración de conceptos vistos en el curso
- Claridad en la comunicación escrita

Aspectos que afectan negativamente la evaluación

- Respuestas descriptivas sin análisis
- Decisiones sin justificación técnica
- Incoherencias entre las partes del sistema
- Falta de estructura en los documentos
- Errores de redacción y ortografía

Criterio de calidad

Un trabajo técnicamente correcto pero sin justificación o con redacción deficiente no podrá obtener la máxima calificación.

Rúbrica de Evaluación:

Criterio	Subcriterio	Puntos	Excelente (100%)	Bueno (80%)	Regular (60%)	Deficiente (30%)	No logrado (0%)
1. Arquitectura del Sistema (20 pts)	Tipo de aplicación	3	Identifica correctamente el tipo de aplicación (nativa, híbrida, etc.) y lo justifica con base en el caso planteado y el contexto del sistema.	Identifica correctamente el tipo de aplicación, pero la justificación es general o poco contextualizada.	Clasificación poco clara o con justificación débil.	Clasificación incorrecta o sin sustento técnico.	No presenta.
	Patrón de arquitectura	4	Selecciona correctamente el patrón (ej. MVVM) y explica cómo aplica al sistema propuesto.	Selecciona correctamente el patrón, pero no explica claramente su aplicación.	Menciona el patrón sin explicar su uso en el sistema.	Selección incorrecta o incoherente.	No presenta.
	Justificación técnica	5	Justificación clara, coherente y directamente relacionada con el problema, considerando escalabilidad y mantenimiento.	Justificación válida pero general o poco profunda.	Justificación superficial sin conexión clara con el sistema.	Justificación incorrecta o incoherente.	No presenta.
	Diagrama de arquitectura	4	Diagrama completo, claro, bien organizado y coherente con la solución planteada.	Diagrama entendible pero con elementos faltantes o poco detalle.	Diagrama básico o difícil de interpretar.	Diagrama incorrecto o incoherente.	No presenta.
	Flujo del sistema	4	Explica correctamente el flujo end-to-end (usuario → app → backend conceptual → respuesta).	Explicación parcial del flujo.	Flujo incompleto o confuso.	Flujo incorrecto.	No presenta.
2. Diseño Backend (15 pts)	Definición de capas	5	Define correctamente Controller, Service y Repository con estructura clara.	Define las capas pero con poca claridad.	Definición incompleta o confusa.	Capas mal definidas o incorrectas.	No presenta.
	Responsabilidades	5	Explica correctamente la responsabilidad de cada capa sin mezclar conceptos.	Explicación parcialmente correcta con leves confusiones.	Mezcla responsabilidades o explica de forma superficial.	Confusión clara entre capas.	No presenta.
	Flujo de petición	5	Describe correctamente el flujo de una petición desde la UI hasta la base de datos.	Flujo parcialmente correcto.	Flujo incompleto.	Flujo incorrecto.	No presenta.
	Definición de endpoints	5	Define endpoints coherentes con	Endpoints válidos pero genéricos.	Endpoints poco claros o incompletos.	Endpoints incorrectos.	No presenta.
3. API Contracts (15 pts)	Definición de endpoints	5	Define endpoints coherentes con	Endpoints válidos pero genéricos.	Endpoints poco claros o incompletos.	Endpoints incorrectos.	No presenta.

Criterio	Subcriterio	Puntos	Excelente (100%)	Bueno (80%)	Regular (60%)	Deficiente (30%)	No logrado (0%)
			el sistema, alineados con funcionalidades reales.				
	Request	5	Estructura completa con campos adecuados, tipos de datos y coherencia.	Request válido pero con campos incompletos.	Request básico o poco claro.	Request incorrecto.	No presenta.
	Response	5	Response completo, coherente con el request y bien estructurado.	Response válido pero incompleto.	Response básico.	Response incorrecto.	No presenta.
4. Interfaz y funcionalidades (Jetpack Compose + Navegación) (30 pts)	Uso de Scaffold	2	Implementa correctamente Scaffold estructurando la pantalla.	Uso parcial.	Uso incorrecto.	Uso mínimo o incorrecto.	No usa.
	Uso de LazyColumn	2	Implementa correctamente listado dinámico.	Uso parcial.	Uso incorrecto.	Deficiente.	No usa.
	Uso de Card/Surface	2	Usa correctamente para representar elementos con diseño adecuado.	Uso básico.	Uso incorrecto.	Deficiente.	No usa.
	Uso de BottomBar	2	Correctamente implementado y visible en ambas pantallas.	Funciona parcialmente.	Implementación incorrecta.	Deficiente.	No usa.
	Layouts + Modifier	2	Uso adecuado de Row/Column/Box y Modifier para diseño claro.	Uso básico.	Uso limitado.	Uso incorrecto.	No usa.
	Separación Screens/Componentes	5	Separación clara y correcta entre pantallas y componentes.	Separación presente pero mejorable.	Separación débil.	No hay separación.	No aplica.
	Reutilización de componentes	5	Componentes reutilizables bien definidos y aplicados.	Reutilización parcial.	Poca reutilización.	No reutiliza.	No aplica.
	Limpieza y legibilidad del código	4	Código limpio, legible, sin redundancia.	Código claro con pequeños problemas.	Código desordenado.	Difícil de leer.	Incorrecto.
	Modularidad	3	Código bien organizado en archivos y funciones.	Organización aceptable.	Organización básica.	Desorganizado.	No aplica.
	Uso de inglés	3	Variables, funciones y archivos en inglés de forma	Algunos errores.	Mezcla idiomas.	Mayormente incorrecto.	No aplica.

Criterio	Subcriterio	Puntos	Excelente (100%)	Bueno (80%)	Regular (60%)	Deficiente (30%)	No logrado (0%)
			consistente.				
5. Diseño de UI (documentación) (10 pts)	Estructura de la UI	3	Explica claramente la estructura de la interfaz y navegación.	Explicación parcial.	Descripción básica.	Incorrecta.	No presenta.
	Definición de composables	3	Identifica y describe correctamente los composables creados.	Parcial.	Incompleto.	Incorrecto.	No presenta.
	Reutilización	2	Identifica correctamente componentes reutilizables.	Parcial.	Débil.	Incorrecto.	No presenta.
	Justificación	2	Justifica decisiones de diseño de UI de forma clara.	Justificación general.	Débil.	Incorrecta.	No presenta.
6. Análisis de la aplicación y propuesta de mejora (10 pts)	Integraciones propuestas	5	Propone integraciones relevantes (ej. notificaciones, pagos, etc.) y las justifica en el contexto del sistema.	Propuestas válidas pero generales.	Propuestas genéricas.	Irrelevantes.	No presenta.
	Mejora de UI	5	Propone mejoras claras basadas en usabilidad, claridad y experiencia de usuario.	Propuestas válidas pero superficiales.	Mejora básica.	Incorrecta.	No presenta.
7. Organización del repositorio (5 pts)	Estructura de carpetas	5	Cumple completamente con la estructura requerida. Todos los archivos en ubicación correcta.	Mínimos errores.	Varias ubicaciones incorrectas.	Desorden general.	No cumple.
8. Redacción y comunicación técnica (5 pts)	Claridad y ortografía	5	Redacción clara, profesional, coherente y sin errores ortográficos.	Pocos errores.	Errores frecuentes.	Mala redacción.	Ininteligible.